

15-Shallow/Deep Copy, Tuples, and Dictionaries in Python

Introduction

- This lecture covers key Python programming concepts including shallow copy, deep copy, aliasing, tuples, and dictionaries, alongside practical classroom Q&A and assignment logistics.

Shallow Copy vs Deep Copy

- **Aliasing:**
Refers to giving a different name to an existing object. If two variables point to the same memory address, changes to one reflect in the other.
- **Shallow Copy:** Created using `list.copy()` or slicing. Produces a new object for the outer list, but nested elements (e.g., lists within lists) retain references, so changes to mutable nested elements reflect across copies.
- **Deep Copy:** Requires `from copy import deepcopy`. It recursively copies all nested elements, so modifications in nested elements of the copied structure do not impact the original.
- **When to use:**
Deep copy is used when you have nested mutable elements and need complete independence between the copies.

Tuple

Properties

- Tuples are immutable collections (their elements cannot be changed after creation).

- Faster and more secure than lists due to immutability—commonly used for data transactions with databases.
- Creation: Use parentheses or simply assign comma-separated values; a single-element tuple requires a trailing comma.
- Tuples can store heterogeneous types, support positive/negative indexing, slicing, concatenation, repetition, and membership operations.
- Can contain duplicate elements.
- Tuple comprehension creates a generator object, not a true tuple.
- Attempting to modify a tuple creates a new object with a new memory address.

Methods

- Only two main methods: `count()` (number of element occurrences), and `index()` (returns the position of first occurrence).

Packing and Unpacking

- **Packing:** Assigning multiple values to one variable (creates a tuple).
- **Unpacking:**
Assigning tuple elements to individual variables. The number of variables must match the tuple's length.
- Clarified via Q&A: Index order of elements is crucial; mismatch in count throws an error; applies to lists for unpacking, not packing.

Dictionary

Properties

- Dictionaries are collections of key-value pairs; each pair is an element (or item).
- Used for structured data, where each value is mapped with a key for easy reference.
- Dictionaries are mutable: changes are made in place (memory address remains the same).
- Keys must be immutable objects (e.g., strings, numbers, tuples); mutable objects like lists cannot be used as keys.
- Keys must be unique; duplicate keys update the value.
- Values can be repeated and can be any object.

Membership & Access

- Membership checks in dictionaries test for keys, not values.
- Values are accessed only through their keys — direct index-based access is not supported.

Edge Cases and Q&A

- Changing a dictionary key creates a new dictionary object, not an in-place change.
- Updating values is in-place, maintaining the memory address.
- Discussion on how mutable/immutable constraints and operations impact dictionary structure.

Assignments and Class Logistics

- Assignment deadlines were discussed and extended to the 20th.
- Class will feature assignment discussion sessions to clarify difficult questions post-submission.
- Attendees informed about file availability issues and coming upcoming topics (dictionary methods, set, frozenset).

Additional Notes

- Throughout, the session featured live Q&A addressing student doubts about object references, effect of operations on memory, operation results, and the application of packing/unpacking and data structures in practice.
 - Instructor provided application use cases and suggestions for performance and security best practices (e.g., choosing tuple over list for certain operations).
-